APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

KTU SPECIAL

We can together dream the B.tech

APJ ABDUL KALAM
TECHNOLOGICAL UNIVERSITY
സാങ്കേതിക ശാസ്ത്ര സർവകലാശാല

• KTU STUDY MATERIALS	• SYLLABUS
• KTU LIVE NOTIFICATION	• SOLVED QUESTION PAPERS

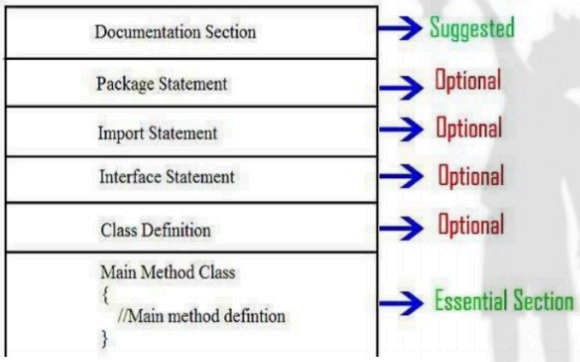
JOIN WITH US

WWW.KTUSPECIAL.IN KTUSPECIAL t.me/ktuspecial1

MODULE -1

PREVIOUS YEAR UNIVERSITY QUESTIONS

1. Describe the programming structure of Java that deals with the organization of Java code



Documentation Section

•You can write a comment in this section. It helps to understand the code. These are optional
•It is used to improve the readability of the program.
•The compiler ignores these comments during the time of execution
•There are three types of comments that Java supports

- Single line Comment **//This is single line comment**
- Multi-line Comment **/* this is multiline comment. and support multiple lines*/**
- Documentation Comment **/** this is documentation cmnt*/**

Package Statement

•We can create a package with any name. A package is a group of classes that are defined by a name.
•That is, if you want to declare many classes within one element, then you can declare it within a package
•It is an optional part of the program, i.e., if you do not want to declare any package, then there will be no problem with it, and you will not get any errors.

- Package is declared as: **package package_name;**

Eg: package mypackage;

Import Statement

• If you want to use a class of another package, then you can do this by importing it directly into your program.
•Many predefined classes are stored in packages in Java
•We can import a specific class or classes in an import statement.
Examples:

- import java.util.Date; //imports the date class
- import java.applet.*; /*imports all the classes from the java

Interface Statement

•This section is used to specify an interface in Java
•Interfaces are like a class that includes a group of method declarations
•It's an optional section and can be used when programmers want to implement **multiple inheritances** within a program

Class Definition

•A Java program may contain several class definitions.
•Classes are the main and essential elements of any Java program.
•A class is a collection of variables and methods

Main Method Class

•The main method is from where the execution actually starts and follows the order specified for the following statements
•This is an essential part of a Java program.
•There may be many classes in a Java program, and only one class defines the main method
•Methods contain data type declaration and executable statements.

Example:

```
public class HelloWorld {
   public static void main(String[] args) {
      System.out.println("Hello, World!");
   }
}
```

- public class HelloWorld: Every Java program must have at least **one class**. Here, it's HelloWorld.Ensure that the class name starts with a capital letter, and the **public** word means it is **accessible from any other classes.**
- public static void main(String[] args): The main method is the entry point of any Java application.
    - When the main method is declared **public**, it means that it can be used outside of this class as well.
    - The word **static** means that we want to access a method without making its objects
    - The word **void** indicates that it does not return any value. The main is declared as void because it does not return any value.
    - **main** is a method; this is a starting point of a Java program.

- **String[] args**: It is an array where each element is a string, which is named as args. If you run the Java code through a console, you can pass the input parameter. The main() takes it as an input.

- System.out.println(...): Prints text to the console. The method **println** prints the text on the screen with a new line. We can also use **print()** method instead of **println()** method. All Java statement ends with a semicolon

2. Why is Java said to be robust?

Robust: Strong memory management and exception handling.

Strong Memory Management

Java utilizes an **automatic garbage collector** which automatically reclaims memory that is no longer being used by the program. This is a significant advantage over languages where developers have to manually manage memory (like C++), as it **prevents memory leaks** and other memory-related errors that can cause applications to crash.

Exception Handling

Java provides a sophisticated and well-defined **exception handling mechanism**. This allows developers to write code that can gracefully **catch and handle errors or exceptional conditions** that might occur during runtime. Instead of the program crashing, it can recover from errors, log them, or take alternative actions, making the application more stable and reliable.

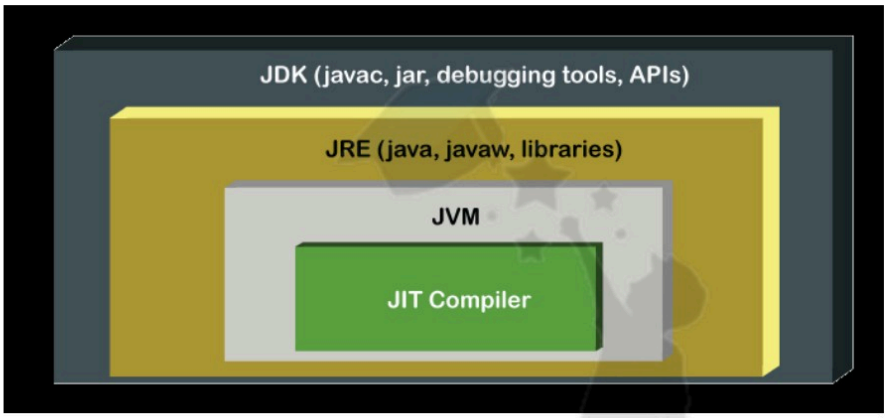
3. Why Java is said to be a secure programming language

Secure: Java includes built-in security features like bytecode verification.

Bytecode Verifier: When Java source code is compiled, it's converted into **bytecode**, which is platform-independent. Before this bytecode is executed by the Java Virtual Machine (JVM), it undergoes a rigorous verification process by the **bytecode verifier**. This verifier checks for code validity, type safety, proper object access, and adherence to Java's security rules. It ensures that the bytecode does not contain any operations that could violate security constraints, such as faking pointers, forging access restrictions, or overflowing the stack. This acts as a crucial barrier against malicious or ill-formed code.

Exception Handling: Java's robust **exception handling** mechanism contributes to security. Instead of crashing when an unexpected error occurs, a well-written Java application can gracefully handle exceptions. This prevents the program from terminating abruptly and potentially leaving the system in an insecure or vulnerable state. Controlled error handling reduces the attack surface for malicious actors.

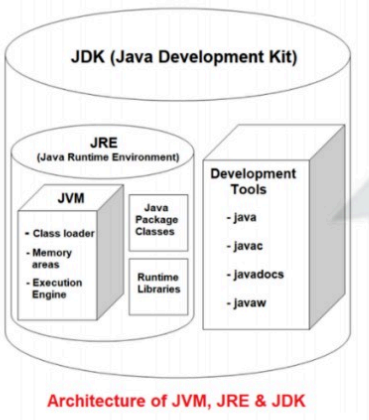
4. Describe the role of JIT compiler



- **JIT** is a part of the JVM that optimizes the performance of the application.
- **JIT** stands for **Java-In-Time Compiler**.
- The **JIT compilation** is also known as dynamic compilation.
- It optimizes the performance of the Java application at compile or run time
- If the JIT compiler environment variable is properly set, the JVM reads the .class file (bytecode) for interpretation after that it passes to the JIT compiler for further process.
- After getting the bytecode, the JIT compiler transforms it into the native code (machine-readable code).
- Java Development Kit (JDK) provides the Java compiler (javac) to compile the Java source code into the bytecode (.class file). After that, JVM loads the .class file at runtime and transform the bytecode into the binary code (machine code).
- Further, the machine code is used by the interpreter

5. Write short notes on JRE,JDK,JVM

- **Java runtime environment (JRE)** provides the software necessary for running Java applications. JRE is an installation package which provides environment to only run(not develop) the java program(or application)onto your machine. JRE is only used by them who only wants to run the Java Programs i.e. end users of your system. JRE can be view as a subset of JDK



Architecture of JVM, JRE & JDK

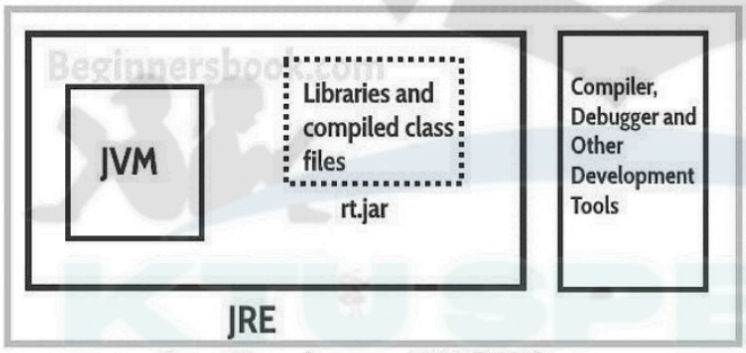
JAVA DEVELOPMENT KIT (JDK)

• The Java Development Kit (JDK) is a software development environment used for developing and executing Java applications and applets

•It includes the

- Java Runtime Environment (JRE),
- an interpreter/loader (Java),
- a compiler (javac),
- an archiver (jar),
- a documentation generator (Javadoc) and other tools needed in Java development.

•JDK is only used by Java Developers



Java Development Kit(JDK)

JAVA VIRTUAL MACHINE (JVM)

•JVM is a program which provides the runtime environment to execute Java programs. Java programs cannot run if a supporting JVM is not available.

•JVM executes the byte code generated by compiler and produce output.

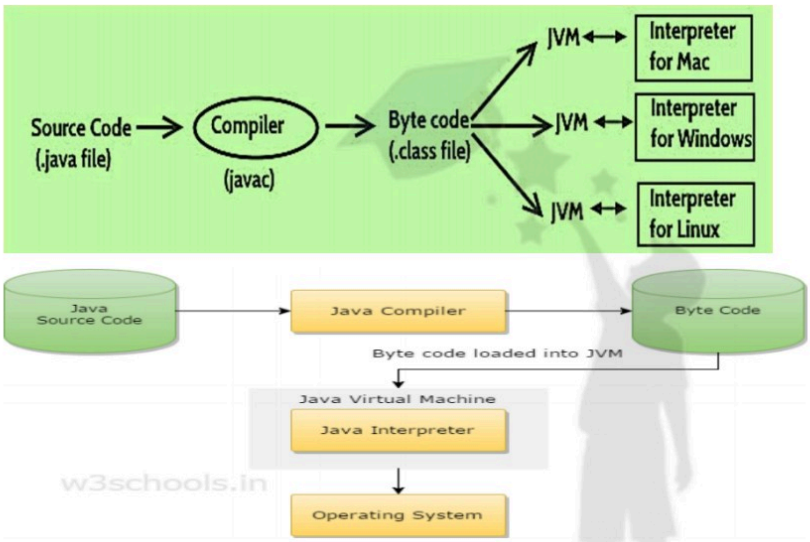
•JVM is the one that makes java platform independent.

•The JVM doesn't understand Java source code, that's why we need to have **javac** compiler

•Java compiler (javac) compiles *.java files to obtain *.class files that contain the byte codes understood by the JVM.

•JVM makes java portable (write once, run anywhere).

•Each operating system has different JVM, however the output they produce after execution of byte code is same across all operating systems.



⚙️ Java Compiler and Java Virtual Machine (JVM)

6. What is the role of Java Virtual Machine?

JAVA VIRTUAL MACHINE (JVM)

•JVM is a program which provides the runtime environment to execute Java programs. Java programs cannot run if a supporting JVM is not available.

•JVM executes the byte code generated by compiler and produce output.

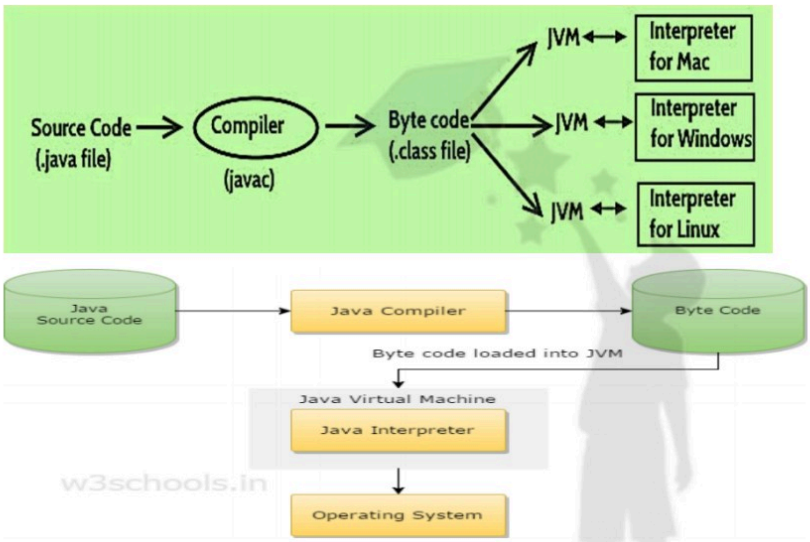
•JVM is the one that makes java platform independent.

•The JVM doesn't understand Java source code, that's why we need to have **javac** compiler

•Java compiler (javac) compiles *.java files to obtain *.class files that contain the byte codes understood by the JVM.

•JVM makes java portable (write once, run anywhere).

•Each operating system has different JVM, however the output they produce after execution of byte code is same across all operating systems.



⚙️ Java Compiler and Java Virtual Machine (JVM)

Java uses a **two-step process** to run programs:

Step 1: Java Compiler (javac)
The **Java compiler** is a program that:

- **Converts** Java source code (.java file) into **bytecode** (.class file).
- **Bytecode** is an intermediate, platform-independent code.

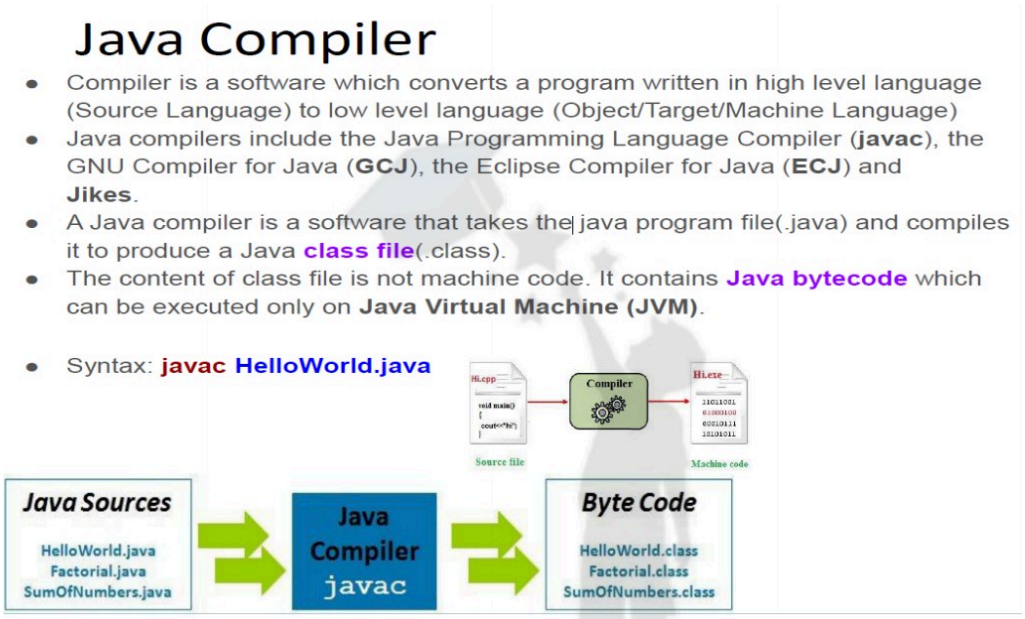
📌 **Example**:
When you write:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Save it as HelloWorld.java.
Now compile it:

```
javac HelloWorld.java
```

This creates a file called HelloWorld.class — which contains **bytecode**.



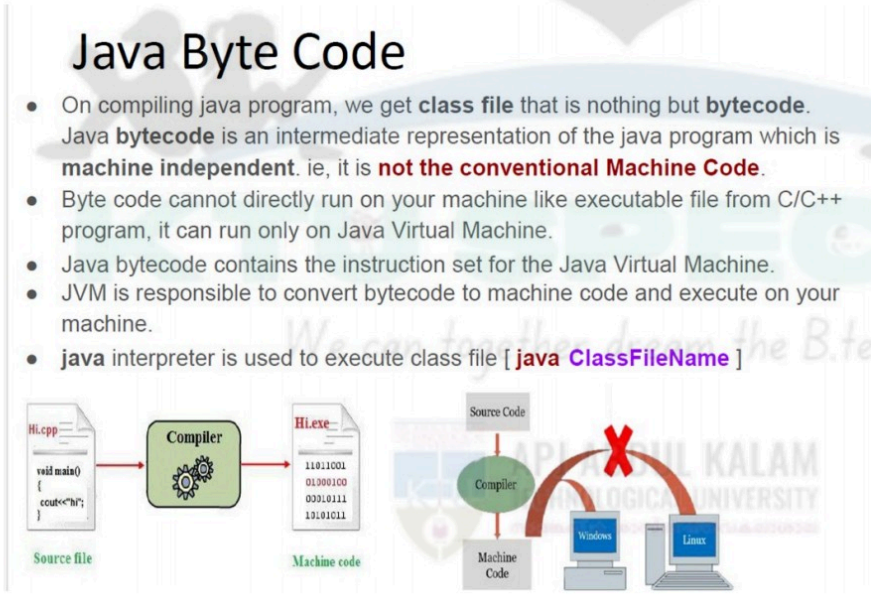
Step 2: Java Virtual Machine (JVM)

The **JVM** is a virtual engine that:

- **Reads** and **executes** the bytecode (.class file).
- Converts bytecode to **machine-specific code** using **Just-In-Time (JIT) compilation**.
- Ensures that the same .class file can run on any device with a JVM (Windows, Mac, Linux, etc.).

📌 **To run the program: java HelloWorld**

The **JVM** runs the bytecode and produces the output: **Hello, World!**



7. Explain implicit casting.

🔹 **1. Widening Type Casting (Automatic)**

Definition: Converting a smaller data type to a larger one.
👉 **Safe conversion** – done automatically by Java.

🔸 **Example:**

```
public class WideningExample {
    public static void main(String[] args) {
        int a = 10;
        double b = a;  // int → double

        System.out.println("int value: " + a);
        System.out.println("double value: " + b);
    }
}
```

Output:

```
int value: 10
double value: 10.0
```

✅ **Widening Order:**

byte → short → int → long → float → double

8. Differentiate between primitive datatype and wrapper data type.

Key Differences

Feature	Primitive Data Types	Wrapper Types
Storage	Stored in stack	Stored in heap
Value	Directly holds value	Holds a reference to an object
Null Value	Cannot have null	Can have null
Memory	More efficient	Less efficient
Usage	Not used in collections	Used in collections
Methods	No methods	Provides methods for operatio

9. Explain how elements are accessed in Java's dynamic array classes .

get(index) -returns an element specified by the index
iterator() -returns an iterator object to sequentially access vector elements

```
import java.util.Iterator;
import java.util.Vector;
class Vectordemo
{
 public static void main(String args[]){
        Vector<String> subject=new Vector<>();
        subject.add("DS");
        subject.add("OOPS");
        subject.add(2,"TOC");
        System.out.println("Vector Elements are "+ subject);
        Vector<String> sub=new Vector<>();
        sub.add("Maths");
        System.out.println("Vector Elements are "+ sub);
        sub.addAll(subject);
        System.out.println("New Vector Elements are "+ sub);
        Iterator<String> s1 = subject.iterator();
        System.out.println("Subjects in the Vector:");
            while (s1.hasNext())
        {
         System.out.println(s1.next());
        }
        String s=subject.get(0);
        System.out.println(" Elements at index 0 "+ s);
        }}
```

10. Outline different methods to insert elements into a vector class.

Add elements to vector

- **add(element) -adds an element to vectors**
- **add(index, element) -adds an element to the specified position**
- **addAll(vector) -adds all elements of a vector to another vector**

11. How memory is allocated in an array

Reference Assignment: After allocation and initialization, the memory address of the newly created array object on the heap is assigned to the array reference variable (e.g., `numbers` in our example).

Example: `int[] numbers = new int[10];`

- A block of memory large enough for 10 integers (e.g., 40 bytes) is allocated on the **heap**.
- Each of these 10 integer slots is initialized to `0`.
- The `numbers` reference variable (on the stack) now points to this allocated block of memory on the heap.

12. How default constructors in java is created

The primary reason for Java providing a default constructor is to ensure that **every object can be instantiated**. If a class had no constructors at all, you wouldn't be able to create objects of that class using the new keyword, making the class unusable. The default constructor provides a basic way to create an object when no specific initialization logic is required by the developer

- <class_name>(){}

```
class Box {
double width;
double height;
double depth;
Box()
 {
System.out.println("Constructing Box");
width = 10;
height = 10;
depth = 10;
}
double volume()
{
double v=width * height * depth;
return  v;
}}
class BoxDemo6 {
public static void main(String args[]) {
Box b1 = new Box();
Box b2 = new Box();
double vol;
```

```
vol = b1.volume();
System.out.println("Volume is " + vol);
// get volume of second box
vol = b2.volume();
System.out.println("Volume is " + vol);
}}
```

13. Differentiate between static and non static method with an example.

```
public class MyClass {
  // Static method
  static void myStaticMethod() {
    System.out.println("Static methods can be called without creating objects");
  }

  // Public method
  public void myPublicMethod() {
    System.out.println("Public methods must be called by creating objects");
  }

  // Main method
  public static void main(String[] args) {
    myStaticMethod(); // Call the static method
    // myPublicMethod(); This would compile an error

    MyClass myObj = new MyClass(); // Create an object of MyClass
    myObj.myPublicMethod(); // Call the public method on the object
  }
}
```

- A `static` method belongs to the **class itself**, not to any specific instance (object) of that class.
- It is declared using the `static` keyword.
- `static` methods **cannot use the `this` or `super` keywords**. This is because `this` refers to the current object instance, and `super` refers to the parent class's instance, neither of which exists in the context of a static method.

❖ A non-static method (also called an **instance method**) belongs to a specific **instance (object)** of the class.

- ❖ It is declared without the static keyword.
- ❖ Non-static methods can use the this and super keywords. this refers to the current object on which the method is called, and super refers to the superclass's instance members.

14. Write a java program to check given number is prime number or not

```
import java.util.Scanner;
class prime
{
public static void main(String args[]){
int n,c=0,i;
Scanner s=new Scanner(System.in);
System.out.println("enter number");
n=s.nextInt();
for(i=1;i<=n;i++){
 if(n%i==0)
{
    c=c+1;
}}
if(c==2){
   System.out.println("given number is prime");
}
else
{
    System.out.println("given number is not prime");

}}}
```

15. Illustrate different primitive data types with an example? Why is that, in Java the size of 'char' datatype is of 2 bytes while that in C is of 1 byte?

Type	Size	Default Value	Description	Example
byte	1 byte (8-bit)	0	Small integers (-128 to 127)	byte a = 10;
short	2 bytes	0	Larger than byte (-32,768 to 32,767)	short b = 1000;
int	4 bytes	0	Common for integers	int c = 12345;
long	8 bytes	0L	Very large integers	long d = 100000L;
float	4 bytes	0.0f	Single-precision decimal	float e = 3.14f;
double	8 bytes	0.0d	Double-precision decimal (default)	double f = 3.14159;
char	2 bytes	'\u0000'	Single 16-bit Unicode character	char g = 'A';
boolean	1 bit (virtual)	false	Logical true/false	boolean h = true;

Java's `char` (2 bytes)

1. **Unicode Support:** Java was designed from its inception to be a truly internationalized language. It uses **Unicode** as its fundamental character set.
2. **UTF-16 Encoding:** Specifically, Java's `char` is a 16-bit (2-byte) unsigned integer type that represents a **UTF-16 code unit**.

C's `char` (1 byte)

1. **ASCII / Local Character Sets:** In C, the `char` data type is typically a **1-byte (8-bit) integer type**. Its primary purpose is to represent characters from the **ASCII character set** or the **local execution character set**.
2. ASCII is a 7-bit encoding, capable of representing 128 characters (primarily English alphabet, numbers, and basic symbols). An 8-bit `char` can represent 256 characters, which is sufficient for many Western European languages

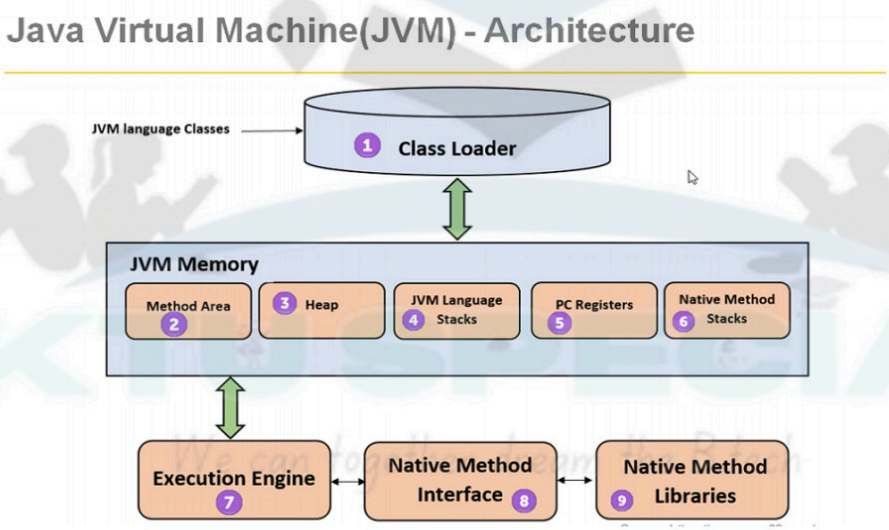
16. Consider a scenario where a class 'Rectangle' with two data members 'Length', 'Breadth' has to be defined and initialized. Sometimes there would be a need that the instance initialization should happen by copying the value from an already initialized instance to the new instance. Model such a class with appropriate constructors and illustrate the working of the class.

```
class Rectangle {

   double length;
   double breadth;
   public Rectangle(double length, double breadth) {
            this.length = length;
      this.breadth = breadth;
      System.out.println("Rectangle created using parameterized constructor (Length: " +
length + ", Breadth: " + breadth + ")");
   }
   public Rectangle(Rectangle originalRectangle) {
      // Copy values from the 'originalRectangle' object to 'this' new object
      this.length = originalRectangle.length;
      this.breadth = originalRectangle.breadth;
      System.out.println("Rectangle created using copy constructor (Copied from another
instance)");
   }
     public double calculateArea() {
      return length * breadth;
```

```
    }
        public void displayDimensions() {
        System.out.println("Dimensions: Length = " + length + ", Breadth = " + breadth);
    }}
class RectangleDemo {
    public static void main(String[] args) {
        System.out.println("\nCreating rectangle1:");
        Rectangle rectangle1 = new Rectangle(10.0, 5.0);
        rectangle1.displayDimensions();
        System.out.println("Area of rectangle1: " + rectangle1.calculateArea());
        System.out.println("\nCreating rectangle2 by copying rectangle1:");
        Rectangle rectangle2 = new Rectangle(rectangle1); // Calls the copy constructor
        rectangle2.displayDimensions();
        System.out.println("Area of rectangle2: " + rectangle2.calculateArea());
          rectangle1.length = 15.0; // Directly modifying the data member for demonstration
        rectangle1.displayDimensions();
         rectangle2.displayDimensions();
    }}
```

17. Illustrate the major functionalities of the 'Class Loader' component within the JVM architecture.



- subsystem used for loading class files.
- It performs three major functions
    - Loading
    - Linking
    - Initialization.

- **Loading** :
    - The Class loader reads the .class file, generate the corresponding binary data and save it in method area.
    - For each .class file, JVM stores following information in method area.
        - Fully qualified name of the loaded class and its immediate parent class.
        - Whether .class file is related to Class or Interface
        - Modifier, Variables and Method information etc.
    - After loading .class file, JVM creates an object of type Class to represent this file in the heap memory.
- **Linking** : Performs verification, preparation
        - *Verification*
            - It ensures the correctness of .class file i.e. it check whether this file is **properly formatted and generated by valid compiler or not**.
            - If verification fails, we get **run-time exception** java.lang.VerifyError.
        - *Preparation*
            - JVM allocates memory for class variables and initializing the memory to default values.
- **Initialization** :
        - In this phase, all **static variables** are assigned with their values defined in the code and static block(if any).
        - This is executed from **top to bottom** in a class and from **parent to child** in class hierarchy.

18. Is it possible to create an object for class A using, A ob = new A(); if the class contains only parameterized constructor? Why?

No, it is **not possible** to create an object for class A using A ob = new A(); if the class A contains **only parameterized constructors**

Constructor Matching: When you use the `new` keyword to create an object (e.g., `new A()`), the Java compiler looks for a constructor in class `A` that matches the arguments you provide within the parentheses.

- In `new A()`, you are implicitly attempting to call a **no-argument constructor** (also known as a default constructor or no-arg constructor).

Java provides a **default (no-argument) constructor ONLY IF** you do not explicitly define **any** constructors (neither no-argument nor parameterized) in your class.

19. Define a Java class having overloaded methods to calculate area of rectangle and circle.

```
public class Area {
   public double calculateArea(double length, double width) {
      return length * width;
   }
   public double calculateArea(double radius) {
            return 3.14 * radius * radius;
   }
   public static void main(String[] args) {
      Area  a = new Area();
      // 1. Calculate Area of a Rectangle
      double rectLength = 10.5;
      double rectWidth = 5.0;
      double rectangleArea = a.calculateArea(rectLength, rectWidth);
      System.out.println("Rectangle Area=" + rectangleArea);

  // 2. Calculate Area of a Circle
      double circleRadius = 7.0;
      double circleArea = a.calculateArea(circleRadius); // Calls (double) method
      System.out.println("Circle Area="+ circleArea);

   }}}
```

20. Describe the following statements in Java.: i. switch and ii. break and continue

The if statement in java, makes selections based on a single true or false condition. But **switch case have multiple choice for selection of the statements**

🗣It is like if-else-if ladder statement

🗣How Java switch works:

• Matching each expression with case

• Once it match, execute all case from where it matched.

• Use break to exit from switch

• Use default when expression does not match with any case

```
 switch (expression) {
case value1:
// statement sequence
break;
case value2:
// statement sequence
break;
.
.
.
case valueN:
// statement sequence
break;
default:
// default statement sequence
}
```

```
import java.util.Scanner;
class calc
{
        public static void main(String args[])
        {
          int a,b,c,ch;
           Scanner in = new Scanner(System.in);
        System.out.println("enter two values");
          a = in.nextInt();
          b= in.nextInt();
        System.out.println("enter choice");
          ch=in.nextInt();
          switch(ch)
          {
      case 1:
        c=a+b;
        System.out.println("Sum="+c);
      break;
        case 2:
        c=a-b;
        System.out.println("Difference="+c);
      break;
        case 3:
        c=a*b;
        System.out.println("Product="+c);
      break;
        case 4:
```

```
        c=a/b;
        System.out.println("Division="+c);
      break;
        case 5:
        c=a%b;
        System.out.println("Modulus="+c);
      break;
        default:
        System.out.println("Invalid choice");
}}}
```

The **Java break statement** is used to break loop or switch statement

- ❖ It breaks the current flow of the program at specified condition
- ❖ When a break statement is encountered inside a loop, the loop is immediately terminated, and the program control resumes at the next statement following the loop.
- ❖ In case of inner loop, it breaks only inner loop.

```
class samplebreak
{
public static void main(String args[]){
        int n=1;
      while(n<=10)
        {
        System.out.println(n);
       if(n==5)
        {
        break;
       }
      n++;
     }
}
```

The **Java continue statement** is used to continue the loop

🗣The continue statement is used in loop control structure when you need to jump to the next iteration of the loop immediately

🗣It continues the current flow of the program and skips the remaining code at the specified condition.

🗣 In case of an inner loop, it continues the inner loop only.

```
class samplecontinue
{
public static void main(String args[])
{
        int n=1;
       while(n<=10)
        {
          if(n==5)
         {
       n++;
         continue;
        }
         System.out.println(n);
       n++;
      }}
```

output

1
2
3
4
6
7
8
9
10

21. Write a program to check whether a string is palindrome or not. The input is to be accepted through command line parameter.

```
public class spalindrome {
   public static void main(String args[]) {
      String str = args[0];
      int flag = 0;
      int len = str.length();
      for (int i = 0; i < len / 2; i++) {
         if (str.charAt(i) != str.charAt(len - i - 1)) {
            flag = 1;
            break;
         }  }
```

```
        if (flag == 0) {
            System.out.println("Palindrome");
        } else {
            System.out.println("Not Palindrome");
        } }}
```

output

java spalindrome malayalam

Palindrome

java spalindrome java

Not Palindrome

22. Briefly explain the primitive data types used in Java

. Primitive Data Types in Java

Java has **8 primitive data types**, which are the most basic types of data built into the language. They are **not objects** and are stored directly in memory for performance.

Type	Size	Default Value	Description	Example
byte	1 byte (8-bit)	0	Small integers (-128 to 127)	byte a = 10;
short	2 bytes	0	Larger than byte (-32,768 to 32,767)	short b = 1000;
int	4 bytes	0	Common for integers	int c = 12345;
long	8 bytes	0L	Very large integers	long d = 100000L;
float	4 bytes	0.0f	Single-precision decimal	float e = 3.14f;
double	8 bytes	0.0d	Double-precision decimal (default)	double f = 3.14159;
char	2 bytes	'\u0000'	Single 16-bit Unicode character	char g = 'A';
boolean	1 bit (virtual)	false	Logical true/false	boolean h = true;

- **byte**: The smallest integer type is byte.
  - This is a signed 8-bit. Variables of type byte are especially useful
  - when you're working with a stream of data from a network or file.
  - when you're working with raw binary data that may not be directly compatible with Java's other built-in types.
  - Byte variables are declared by use of the byte keyword.
  - For example, the following declares two byte variables called b and c:

```
byte b, c;
```

- **short** is a signed 16-bit type.
  - you can use a short to save memory in large arrays, in situations where the memory savings actually matters.
  - Example : **short** s;
- **int** :The most commonly used integer type is int.
  - In addition to other uses, variables of type int are commonly employed to control loops and to index arrays.
  - Example: **int** a;
- **long** is a signed 64-bit type and is useful for those occasions where an int type is not large enough to hold the desired value.
  - Example: **long** a;

Floating point

- **Floating-point numbers**, also known as real numbers, are used when evaluating expressions that require fractional precision.
- The type **float** specifies a single-precision value that uses 32 bits of storage.
- Variables of type float are useful when you need a fractional component, but don't require a large degree of precision.
- Example: **float** highTemp, lowTemp;
- **Double precision**, as denoted by the **double** keyword, uses 64 bits to store a value.
- When you need to maintain accuracy over many iterative calculations, or are manipulating large-valued numbers, double is the best choice.
- Example : **double** pi, r, a;

Characters

- In Java, the data type used to store characters is **char**.
- Java uses Unicode to represent characters
- At the time of Java's creation, Unicode required 16 bits. Thus, in Java char is a 16-bit type.
- Example: char letterA= 'A';

Boolean

- Java has a primitive type, called boolean, for logical values.
- It can have only one of two possible values, true or false.
- This is the type returned by all relational operators, as in the case of a < b.
- Example: boolean b;

23. Discuss any two-character extraction methods used for string processing in Java

CHARACTER EXTRACTION

charAt( )

- To extract a single character from a String, you can refer directly to an individual character via the charAt( ).It has this general form: char charAt(int where) .Here, where is the index of the character that you want to obtain.
-

```
public class CharAtExample {

    public static void main(String[] args) {
        String name= "UNIVERSITY"
        char ch=name.charAt(4);//returns the char value at the 4th index
        System.out.println(ch);
```

getChars( )

- if you need to extract more than one character at a time, you can use the getChars( ) method.
- void getChars(int *sourceStart, int sourceEnd, char target[ ], int targetStart)*

```
public class GetCharsDemo {
    public static void main(String args[]) {
        String s = "This is a demo of the getChars method.";
        int start = 10;
        int end = 14;
        char buf[] = new char[end - start];
        s.getChars(start, end, buf, 0);
        System.out.println(buf);
    }
}
```

- getBytes( )

24. What are parameterized constructors? Is it possible to define a parameterized constructor for a class without defining a parameter-less constructor?

Constructor Parameters (Parameterized constructor)

```
/* Here, Box uses a parameterized constructor to
   initialize the dimensions of a box.
*/
class Box {
  double width;
  double height;
  double depth;

  // This is the constructor for Box.
  Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
  }

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}

class BoxDemo7 {
  public static void main(String args[]) {


    // declare, allocate, and initialize Box objects
    Box mybox1 = new Box(10, 20, 15);
    Box mybox2 = new Box(3, 6, 9);

    double vol;

    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume is " + vol);

    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume is " + vol);
  }
}
```

A **parameterized constructor** is a special type of constructor in Java that accepts one or more arguments (parameters). Its primary purpose is to **initialize the instance variables (fields)** of an object with the values provided as arguments during the object's creation

Yes, It is possible to define a parameterized constructor for a class without defining a parameter-less constructor?

25. Explain the terms: Polymorphism and Encapsulation

POLYMORPHISM in Java

📌 Definition:

Polymorphism means “many forms”. In Java, it allows objects to behave differently based on the context, even if they share the same interface or method name.

There are two types:

🔸 **1. Compile-time Polymorphism (Method Overloading)**

- Same method name with **different parameters** (type or number).
- Resolved **at compile time**.

🔸 **2. Runtime Polymorphism (Method Overriding)**

- Same method name and signature in both superclass and subclass.
- Resolved **at runtime** using **dynamic method dispatch**.

Encapsulation is the concept of **binding data and methods that operate on that data into a single unit (class)** and restricting direct access to some of the object's components.

🎯 **Purpose:**

- To **protect** data from unauthorized access.
- To allow **controlled access** through **getters and setters**.
- The wrapping up of data(variables) and function (methods) into a single unit (called class) is known as encapsulation.
- It is also called "information hiding

26. Write a Java program that accepts N integers through console and compute their average.

```
import java.util.Scanner;
public class SimpleAverage {
   public static void main(String[] args) {
                int sum = 0;
      Scanner scanner = new Scanner(System.in);
      System.out.print("How many integers do you want to average? ");
      int N = scanner.nextInt(); // Read that number
        System.out.print("Enter values ");
```

```
for (int i = 0; i < N; i++)
  {
    int num = scanner.nextInt(); // Read each number
    sum += num; // Add it to the total sum
}
double average = (double) sum / N;
System.out.println("The average is: " + average);
 }}
```

27. Discuss about bitwise, relational and conditional operators in Java with examples and compare its precedence

bitwise operators

Operator	Description
& (bitwise and)	Bitwise AND operator give true result if both operands are true. otherwise, it gives a false result.
| (bitwise or)	Bitwise OR operator give true result if any of the operands is true.
^ (bitwise XOR)	Bitwise Exclusive-OR Operator returns a true result if both the operands are different. otherwise, it returns a false result.
~ (bitwise compliment)	Bitwise One's Complement Operator is unary Operator and it gives the result as an opposite bit.
<< (left shift)	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.
>> (right shift)	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.
>>> (zero fill right shift)	Shift right zero fill operator. The left operands value is moved right by the number of bits specified by the right operand and shifted values are filled up with zeros.

, relational operators

Operators	Descriptions	Examples
== (equal to)	This operator checks the value of two operands, if both are **equal**, then it returns true otherwise false.	(2 == 3) is not true.
!= (not equal to)	This operator checks the value of two operands, if both are **not equal**, then it returns true otherwise false.	(4 != 5) is true.
> (greater than)	This operator checks the value of two operands, if the left side of the operator is **greater**, then it returns true otherwise false.	(5 > 56) is not true.
< (less than)	This operator checks the value of two operands if the left side of the operator is **less**, then it returns true otherwise false.	(2 < 5) is true.
>= (greater than or equal to)	This operator checks the value of two operands if the left side of the operator is **greater or equal**, then it returns true otherwise false.	(12 >= 45) is not true.
<= (less than or equal to)	This operator checks the value of two operands if the left side of the operator is **less or equal**, then it returns true otherwise false.	(43 <= 43) is true.

conditional operators

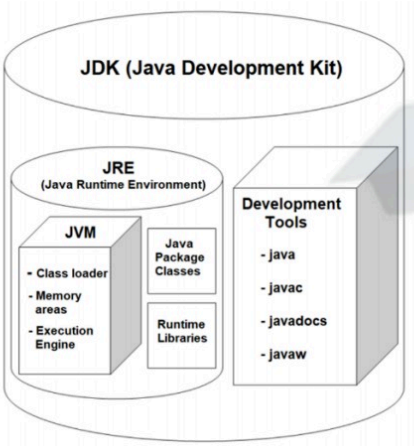
(? :) Expression1 ? Expression2 : Expression3

Expression ? value if true : value if false

28. Illustrate the Java Programming and Runtime Environment. Explain the roles of each component of it while compiling and executing a java program

Java programming Environment and Runtime Environment

- The **Java programming environment** provides tools and libraries for developers to write, compile, and debug Java code.
- **Java runtime environment (JRE)** provides the software necessary for running Java applications. JRE is an installation package which provides environment to only run(not develop) the java program(or application)onto your machine. JRE is only used by them who only wants to run the Java Programs i.e. end users of your system. JRE can be view as a subset of JDK



Architecture of JVM, JRE & JDK

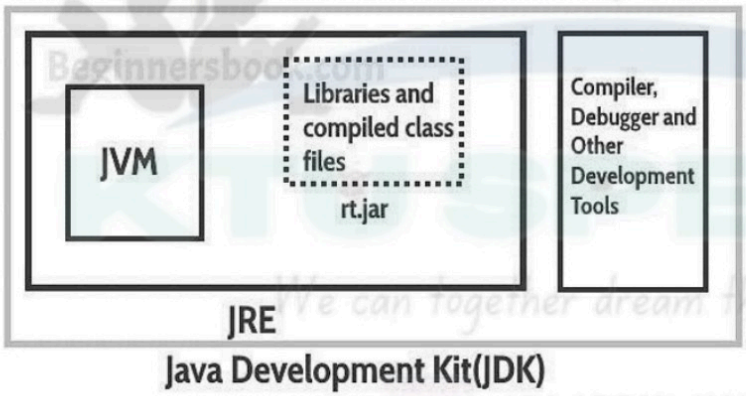
JAVA DEVELOPMENT KIT (JDK)

• The Java Development Kit (JDK) is a software development environment used for developing and executing Java applications and applets

•It includes the

- Java Runtime Environment (JRE),
- an interpreter/loader (Java),
- a compiler (javac),
- an archiver (jar),
- a documentation generator (Javadoc) and other tools needed in Java development.

•JDK is only used by Java Developers



JAVA VIRTUAL MACHINE (JVM)

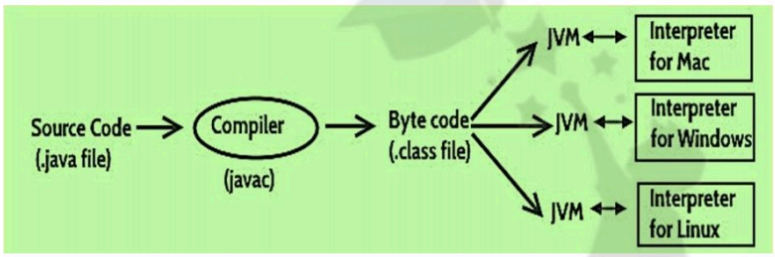
•JVM is a program which provides the runtime environment to execute Java programs. Java programs cannot run if a supporting JVM is not available.

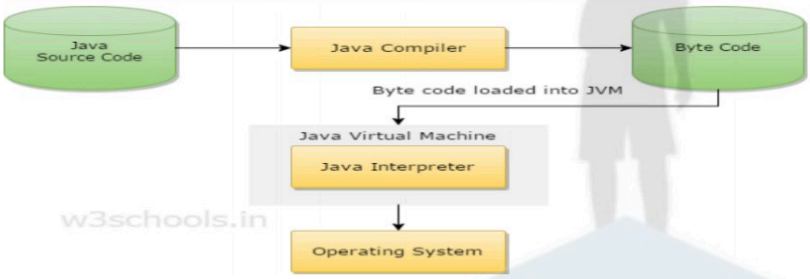
•JVM executes the byte code generated by compiler and produce output.

•JVM is the one that makes java platform independent.

•The JVM doesn't understand Java source code, that's why we need to have **javac** compiler

•Java compiler (javac) compiles *.java files to obtain *.class files that contain the byte codes understood by the JVM.
•JVM makes java portable (write once, run anywhere).
•Each operating system has different JVM, however the output they produce after execution of byte code is same across all operating systems.





⚙️ Java Compiler and Java Virtual Machine (JVM)

Java uses a **two-step process** to run programs:

Step 1: Java Compiler (javac)

The **Java compiler** is a program that:

- **Converts** Java source code (.java file) into **bytecode** (.class file).
- **Bytecode** is an intermediate, platform-independent code.

📌 Example:

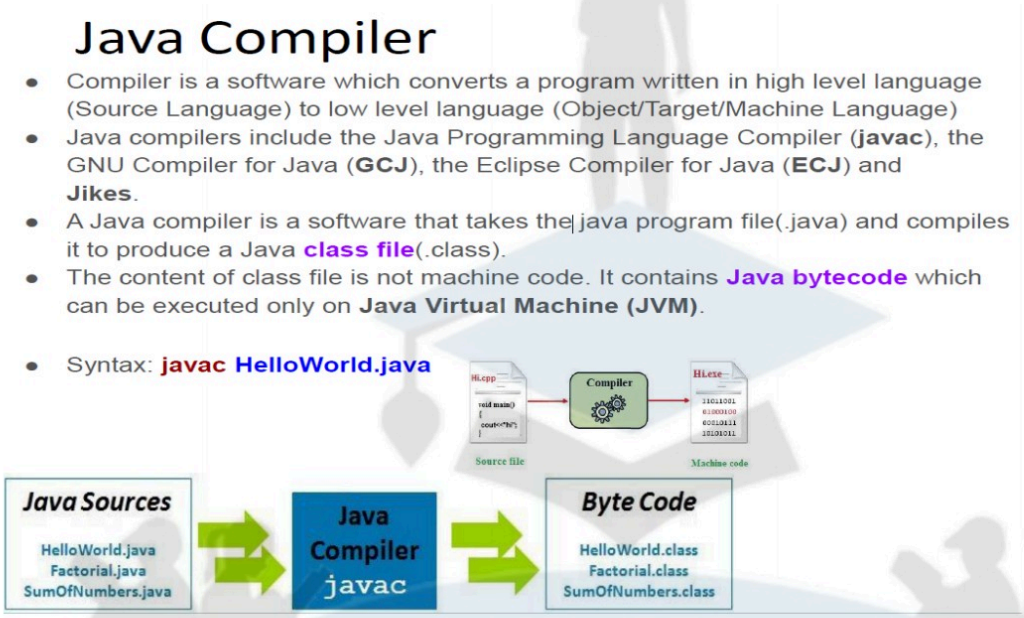
When you write:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Save it as HelloWorld.java.
Now compile it:

```
javac HelloWorld.java
```

This creates a file called HelloWorld.class — which contains **bytecode**.



Step 2: Java Virtual Machine (JVM)

The **JVM** is a virtual engine that:

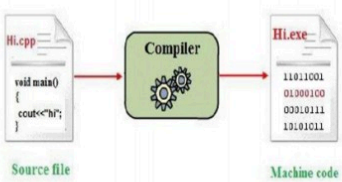
- **Reads** and **executes** the bytecode (.class file).
- Converts bytecode to **machine-specific code** using **Just-In-Time (JIT) compilation**.
- Ensures that the same .class file can run on any device with a JVM (Windows, Mac, Linux, etc.).

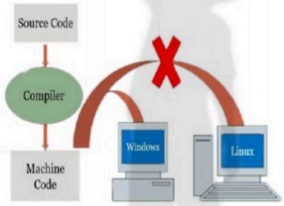
📌 **To run the program: java HelloWorld**

The **JVM** runs the bytecode and produces the output: **Hello, World!**

Java Byte Code

- On compiling java program, we get **class file** that is nothing but **bytecode**. Java **bytecode** is an intermediate representation of the java program which is **machine independent**. ie, it is **not the conventional Machine Code**.
- Byte code cannot directly run on your machine like executable file from C/C++ program, it can run only on Java Virtual Machine.
- Java bytecode contains the instruction set for the Java Virtual Machine.
- JVM is responsible to convert bytecode to machine code and execute on your machine.
- **java** interpreter is used to execute class file [**java ClassFileName**]



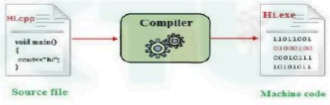


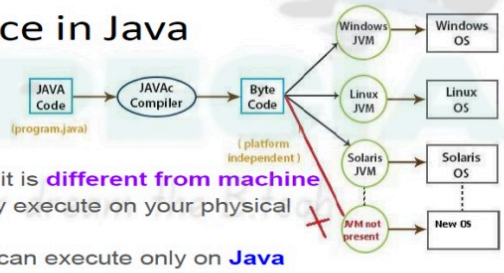
Why is this important?

Component	Role	Output
javac	Java Compiler	.class bytecode file
java (JVM)	Executes the bytecode on your machine	Program output on console

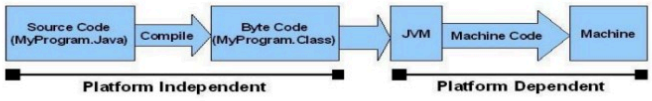
- Java Compiler (javac) ➜ Converts .java to .class (bytecode).
- JVM (java) ➜ Runs the bytecode, making Java **platform-independent**

Platform independence in Java





- Java compiler output is class file and it is **different from machine code**. Only machine code can directly execute on your physical machine.
- Class file contains **byte code**, which can execute only on **Java Virtual Machine(JVM)**.
- Java is platform independent because java class files **can run on all operating systems** without any modifications.
- If JVM is available for a platform, then Java can run on that platform



29. Illustrate the working of any two methods of String class that compare string

STRING COMPARISON

- There are three ways to compare string in java:
  - By equals() method
  - By = = operator
  - By compareTo() method

equals()

The String equals() method compares the original content of the string.

```
class Teststringcomparison1{
 public static void main(String args[]){
   String s1="Sachin";
   String s2="Sachin";
   String s3=new String("Sachin");
   String s4="Saurav";
   System.out.println(s1.equals(s2));//true
   System.out.println(s1.equals(s3));//true
   System.out.println(s1.equals(s4));//false
 }
}
```

```
class Teststringcomparison2{
 public static void main(String args[]){
   String s1="Sachin";
   String s2="SACHIN";

   System.out.println(s1.equals(s2));//false
   System.out.println(s1.equalsIgnoreCase(s2));//true
 }
}
```

```
false
true
```

== operator

The = = operator compares references not values

```
class Teststringcomparison3{
 public static void main(String args[]){
   String s1="Sachin";
```

compareTo()

- compareTo() method compares values lexicographically and returns an integer value that describes if first string is less than, equal to or greater than second string.

- Suppose s1 and s2 are two string variables. If:

  s1 == s2 : 0
  s1 > s2 : positive value
  s1 < s2 : negative value

```
class Teststringcomparison4{
 public static void main(String args[]){
  String s1="Sachin";
  String s2="Sachin";
  String s3="Ratan";
  System.out.println(s1.compareTo(s2));//0
  System.out.println(s1.compareTo(s3));//1(because s1>s3)
  System.out.println(s3.compareTo(s1));//-1(because s3 < s1 )
 }
}
```

```
Output:0
       1
       -1
```

	ARRAY	VECTOR
Feature	Java Arrays (type[] arrayName)	java.util.Vector Class (java.util.Vector<E>)
Size	Fixed-size: Size is set at creation and cannot be changed.	Dynamic / Resizable: Can grow or shrink automatically as needed.
Data Types Stored	Can store both primitive types (e.g., int, char) and objects.	Can only store objects. Primitives are autoboxed.
Memory Allocation	Elements are stored in a contiguous block of memory.	Elements are stored in an internal array, which is re-allocated and copied upon resizing.
Performance	Faster random access (O(1)) due to direct memory addressing. Slower for insertions/deletions in middle (O(n)).	Slower than ArrayList and raw arrays due to synchronization overhead. Random access is O(1) but with more overhead. Insertions/deletions/resizing can be O(n).
Generics Support	Inherently type-safe at declaration (e.g., String[] only holds Strings).	Supports generics (Vector<E>) since Java 5, providing compile-time type safety. Before, it stored Object.

Methods	Basic array syntax (array[index], array.length).	Rich set of methods for collection manipulation (add(), remove(), get(), size(), iterator(), etc.).
Flexibility	Less flexible due to static size.	More flexible for variable-sized collections.
Typical Use Cases	When collection size is known and fixed, and high-speed random access is critical.	In old codebases, or extremely niche cases where the specific Vector synchronization model is desired over ArrayList with explicit synchronization.

31. Write a Java program to find the frequency (count the occurrence) of each element in an integer array.

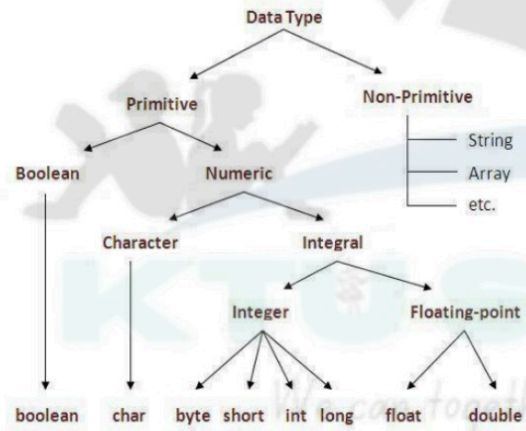
```
public class intfreq {
   public static void main(String[] args) {
        int[] numbers = {10, 20, 10, 30, 40, 20, 10, 50, 30, 10, 60, 20};
       System.out.println("Original array:");
     // Simple loop to print the array for reference
     for (int i = 0; i < numbers.length; i++) {
        System.out.println(numbers[i]);
     }
              for (int i = 0; i < numbers.length; i++) {
                boolean c= false;
        for (int k = 0; k < i; k++) {
          if (numbers[i] == numbers[k]) {
           c = true; // Yes, it appeared before
            break;
          }}
             if (!c) {
          int currentNumber = numbers[i];
          int count = 0;
            for (int j = 0; j < numbers.length; j++) {
             if (numbers[j] == currentNumber) {
                count++; // Found a match
             }}
                      System.out.println("Element " + currentNumber + ": " + count + " times");
     }}}}
```

32. Write a Java program to reverse bits of a given integer

```
class bitreverse{
```

```
    static int reverseBits(int n) {
        int ans = 0;
        while (n > 0) {
          ans <<= 1;
                  if ((n & 1) == 1)
              ans =ans| 1;
                  n >>= 1;
        }
        return ans;
    }
    public static void main(String[] args) {
        int n = 11;
        String binaryString1 = Integer.toBinaryString(n);
        System.out.println("The binary representation of " + n + " is: " + binaryString1);
          int r=reverseBits(n);
          String binaryString2 = Integer.toBinaryString(r);
        System.out.println("The binary representation of " + r + " is: " + binaryString2);
}}
```

33. Explain different data types in Java. Give examples each?



Type	Size	Default Value	Description	Example
byte	1 byte (8-bit)	0	Small integers (-128 to 127)	byte a = 10;
short	2 bytes	0	Larger than byte (-32,768 to 32,767)	short b = 1000;
int	4 bytes	0	Common for integers	int c = 12345;
long	8 bytes	0L	Very large integers	long d = 100000L;
float	4 bytes	0.0f	Single-precision decimal	float e = 3.14f;
double	8 bytes	0.0d	Double-precision decimal (default)	double f = 3.14159;
char	2 bytes	'\u0000'	Single 16-bit Unicode character	char g = 'A';
boolean	1 bit (virtual)	false	Logical true/false	boolean h = true;

34. Why is the 'main' method in Java qualified as public, static, and void?
    - public static void main(String[] args): The main method is the entry point of any Java application.
    - When the main method is declared public, it means that it can be used outside of this class as well.
    - The word static means that we want to access a method without making its objects
    - The word void indicates that it does not return any value. The main is declared as void because it does not return any value.
    - main is a method; this is a starting point of a Java program.
    - String[] args: It is an array where each element is a string, which is named as args. If you run the Java code through a console, you can pass the input parameter. The main() takes it as an input.

35. Write a java program that illustrates how this keyword can be used to resolve the ambiguity between formal parameters and instance variables

```
class Box {
    double width;
    double height;
    double depth;
    public double calculateVolume() {
        return width*height*depth;

    }
    public void setDimention(double width, double height, double depth) {
        this.width = width;
        this.height = height;
        this.depth = depth;
    }
}
// This class declares an object of type Box.
class BoxDemo {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        Box mybox2 = new Box();

        //assign values to mybox's instance variables
        mybox1.setDimention(10, 20, 15);

        //assign values to mybox's instance variables
        mybox2.setDimention(50, 70, 55);

        double volumeOfBox1 = mybox1.calculateVolume();
        System.out.println("Volume of Box1="+volumeOfBox1);
        double volumeOfBox2 =mybox2.calculateVolume();
        System.out.println("Volume of Box1="+volumeOfBox2);
    }
}
```

Or

```
public class Product {
    String name; // Instance variable
    double price; // Instance variable
    // Constructor with formal parameters having the same names as instance variables
    public Product(String name, double price) {
```

```
        // Without 'this', 'name = name;' would assign the parameter 'name' to itself.
        // 'this.name' explicitly refers to the instance variable 'name'.
        this.name = name;
        this.price = price; // 'this.price' explicitly refers to the instance variable 'price'.
    }
    // Method to display product details
    public void displayProductDetails() {
        System.out.println("Product Name: " + this.name);
        System.out.println("Product Price: $" + this.price);
    }
    public static void main(String[] args) {
        // Create a Product object
        Product laptop = new Product("Dell XPS 15", 1500.00);
        // Display product details
        laptop.displayProductDetails();
    }}
```

36. List any two different access specifiers used in java?

- **`public`**
  - **Visibility:** Accessible from **anywhere** (from any class in any package).
  - Often used for methods that define the public interface of a class, or for classes that need to be accessible globally.
- **`private`**
  - **Visibility:** Accessible **only within the class** where it is declared.
  - Primarily used for fields (instance variables) and helper methods to encapsulate data and hide implementation details, adhering to the principle of information hiding.

37. Name the six integer types in Java and outline the bitwise operators that can be applied to the integer types

- **`byte`**:

- **Size:** 8-bit (1 byte)
- **Range:** -128 to 127

`short`:

- **Size:** 16-bit (2 bytes)
- **Range:** -32,768 to 32,767

`int`:

- **Size:** 32-bit (4 bytes)
- **Range:** −231 to 231−1 (approx. -2 billion to +2 billion)

`long`:

- **Size:** 64-bit (8 bytes)
- **Range:** −263 to 263−1 (a very large range, approx. -9 quintillion to +9 quintillion)

char	2 bytes	'\u0000'	Single 16-bit Unicode character	char g = 'A';
boolean	1 bit (virtual)	false	Logical true/false	boolean h = true;

the bitwise operators available in Java:

1. **Bitwise AND (`&`)**:
    - **Description:** Compares each bit of two operands. If both corresponding bits are `1`, the result bit is `1`; otherwise, it's `0`.
    - **Example:**
        - `5` (binary `0101`)
        - `3` (binary `0011`)
        - `5 & 3` = `0001` (decimal `1`)
2. **Bitwise OR (`|`)**:
    - **Description:** Compares each bit of two operands. If at least one of the corresponding bits is `1`, the result bit is `1`; otherwise, it's `0`.
    - **Example:**
        - `5` (binary `0101`)
        - `3` (binary `0011`)
        - `5 | 3` = `0111` (decimal `7`)
3. **Bitwise XOR (`^`)**:

- **Description:** Compares each bit of two operands. If the corresponding bits are **different**, the result bit is `1`; otherwise, it's `0`.
- **Example:**
  - `5` (binary `0101`)
  - `3` (binary `0011`)
  - `5 ^ 3` = `0110` (decimal `6`)

4. **Bitwise Complement (NOT) (~)**:
   - **Description:** A unary operator that inverts all the bits of its operand (changes `0`s to `1`s and `1`s to `0`s). For `int` and `long` types, due to two's complement representation, `~x` is equivalent to `(-x - 1)`.
   - **Example:**
     - `5` (32-bit binary: `0...0101`)
     - `~5` = `1...1010` (32-bit binary), which is `-6` in decimal.
5. **Left Shift (<<)**:
   - **Description:** Shifts the bits of the left-hand operand to the left by the number of positions specified by the right-hand operand. Zeros are shifted in from the right. This effectively multiplies the number by 2n (where n is the shift amount) for non-negative numbers.
   - **Example:**
     - `5` (binary `0000 0101`)
     - `5 << 2` = `0001 0100` (decimal `20`)
6. **Signed Right Shift (>>)**:
   - **Description:** Shifts the bits of the left-hand operand to the right by the number of positions specified by the right-hand operand. The leftmost bits are filled with the **sign bit** (0 for positive numbers, 1 for negative numbers) to preserve the original number's sign. This effectively divides the number by 2n for non-negative numbers.
   - **Example:**
     - `20` (binary `0001 0100`)
     - `20 >> 2` = `0000 0101` (decimal `5`)
     - `-20` (binary `1110 1100`)

- `-20 >> 2` = `1111 1011` (decimal `-5`)

38. State the uses of this keyword.

- **To refer to the current class instance variable (field):** Used to differentiate between instance variables and local variables/parameters with the same name.
- **To invoke (call) the current class constructor:** Used as `this()` (with appropriate arguments) to call another constructor from within the same class (constructor chaining). Must be the first statement in the constructor.
- **To pass the current object as an argument in a method call:** Used to send a reference of the current object to another method.
- **To return the current class instance from a method:** Used to return `this` from a method, enabling method chaining

39. Define a Constructor? State its properties?

Let's define a constructor and then outline its key properties in Java.

What is a Constructor?

A **constructor** is a special type of method in Java that is used to **initialize the state (instance variables) of an object** immediately after it has been created using the new keyword.

Properties of a Constructor:

1. **Same Name as the Class:** A constructor must always have the exact same name as the class it belongs to.
   - *Example:* If the class name is `Car`, its constructor will be `Car()`.
2. **No Return Type:** Constructors do not have any return type, not even `void`. If you specify a return type, it will be treated as a regular method, not a constructor.
3. **Automatic Invocation:** Constructors are automatically called by the Java Virtual Machine (JVM) when an object is created using the `new` operator. You cannot call a constructor explicitly like a regular method (e.g., `myObject.Car()`).
4. **Purpose of Initialization:** Their primary purpose is to perform initialization tasks for the new object, such as assigning initial values to instance variables, setting up resources, etc.
5. **Access Modifiers:** Constructors can have any access modifier (`public`, `private`, `protected`, or `default`/package-private).
   - `public`: Most common, allows object creation from anywhere.
   - `private`: Prevents direct object creation from outside the class (used in patterns like Singleton).
6. **Constructor Overloading:** A class can have multiple constructors, provided each has a different number or type of parameters. This is known as constructor overloading, allowing objects to be initialized in different ways.
7. **No Inheritance:** Constructors are not inherited by subclasses. A subclass must define its own constructors. However, a subclass constructor implicitly or explicitly calls a superclass constructor using `super()`.

8. **Default Constructor (Implicit Constructor):** If a class does not define any constructors explicitly, the Java compiler automatically provides a public, no-argument (parameter-less) constructor. This is called the default constructor.
    - *Important:* If you define *any* constructor (even a parameterized one), the compiler will *not* provide the default constructor.
9. **`this()` and `super()` Calls:**
    - `this()`: Used to call another constructor within the same class (constructor chaining). It must be the first statement in the constructor.
    - `super()`: Used to call a constructor of the superclass. It also must be the first statement in the constructor (if present).

40. Explain how bye code helps in achieving platform independence

Intermediate Code: Java source code (`.java`) is compiled by the Java compiler (`javac`) into an intermediate, machine-independent format called **bytecode** (`.class`).

JVM as Interpreter/Translator: This bytecode is then executed by the **Java Virtual Machine (JVM)**. A specific JVM is available for each different operating system and hardware platform.

"Write Once, Run Anywhere": The JVM acts as a translator, converting the generic bytecode into the specific machine code understandable by the underlying platform. This allows Java programs to be written and compiled once, and then run on any system with a compatible JVM, achieving platform independence.

41. Describe in detail Object Oriented Programming principles. Illustrate with suitable examples.

OBJECT

- Objects are real-world entities that has their own properties and behavior.
- It has physical existence
- Eg: person, banks, company, customers etc

CLASS

- A class is a blueprint or prototype from which objects are created
- A class is a generalized description of an object.
- An object is an instance of a class

Abstraction

- Abstraction means displaying only essential information and hiding the details.

- Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.

Encapsulation

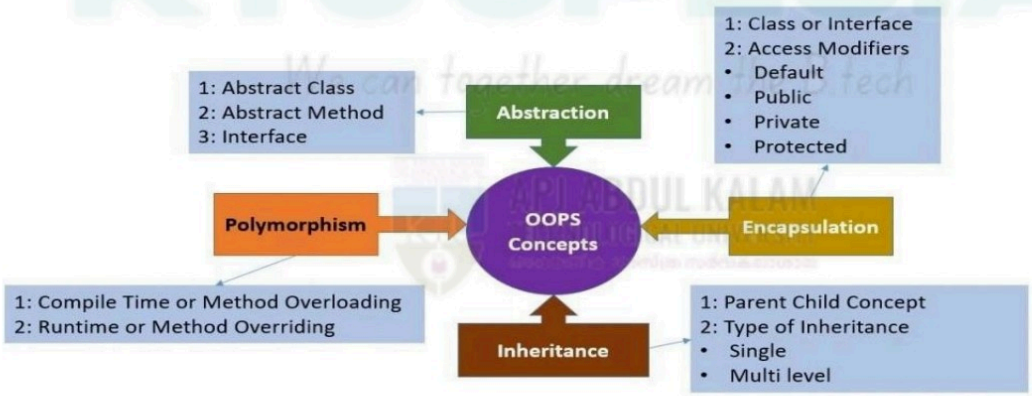
- The wrapping up of data(variables) and function (methods) into a single unit (called class) is known as encapsulation.
- It is also called "information hiding

Inheritance

- The capability of a class to derive properties and characteristics from another class is called Inheritance.
- Inheritance is the process by which objects of one class acquired the properties of objects of another classes
- Sub Class : The class that inherits properties from another class is called Sub class or Derived Class.
- Super Class : The class whose properties are inherited by sub class is called Base Class or Super class

Polymorphism

- means having many forms
- In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form.



42. Explain minimum two salient features of procedural programming and object-oriented programming paradigm approach.

Procedural Programming (PP)

Procedural Programming (PP)

1. **Procedure-Centric:** Programs are structured as a sequence of instructions or functions (procedures) that operate on data. The emphasis is on the "steps to perform" a task.
2. **Separate Data and Functions:** Data is typically kept separate from the functions that process it, and data can often be global, accessible by multiple functions.

Object-Oriented Programming (OOP)

1. **Objects and Classes:** Programs are designed around "objects," which are instances of "classes." Each object bundles both data (attributes) and the methods (functions) that operate on that data.
2. **Encapsulation:** This principle combines data and the methods that operate on it into a single unit (a class), and often restricts direct external access to the data, promoting information hiding.

43. Object-oriented programming uses classes and objects. What are classes and what are objects? What is the relationship between classes and objects?

A **class** is a **blueprint** or **template** that defines the structure and behavior (data and methods) that the objects created from it will have.

- A **design** for creating objects.
- It contains **fields** (variables) and **methods** (functions).

- **Syntax:**

```
class ClassName {
   // Fields (variables)
   // Methods (functions)
}
```

An **object** is an **instance** of a class.

- It represents a **real-world entity**.
- Each object has its **own state** (fields) and **can perform actions** (methods).

- **Example:**

If Car is a class, then **myCar** and **yourCar** are objects of that class.

✅ **Java Program: Creating and Using a Class and Object**

```
// Define a class
class Student {
    // Fields (data members)
    String name;
    int age;
    // Method (behavior)
    void displayDetails() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

// Main class
public class Main {
    public static void main(String[] args) {
        // Create an object of Student class
        Student s1 = new Student();
        // Set values
        s1.name = "Anjali";
        s1.age = 20;
        // Call method
        s1.displayDetails();
    }
}
```

✅ **Output:**

Name: Anjali

Age: 20

- **Key Terminologies**

Term	Description
class	Keyword to define a class
new	Keyword used to create an object
Dot operator.	Used to access members of an object (e.g., object.method())

Write a java program to find the sum of two complex numbers. Use suitable constructors and methods.

44. Write a java program to find the product of two matrices?

```
import java.util.Scanner;
class Test{

public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the order - m1:"); int m1 =
        sc.nextInt(); System.out.print("Enter the order -
        n1:"); int n1 = sc.nextInt(); System.out.print("Enter
        the order - m2:"); int m2 = sc.nextInt();
        System.out.print("Enter the order - n2:"); int n2 =
        sc.nextInt();
        if(n1 != m2){
        System.out.println("Matrix Multiplication not Possible"); return;
        }
        int A[][] = new int[m1][n1];
        int B[][] = new int[m2][n2];
        int C[][] = new int[m1][n2];
        System.out.println("Read Matrix A"); for(int
        i=0;i<m1;i++){
        for(int j=0;j<n1;j++){
        System.out.print("A["+i+"]["+j+"]="); A[i][j] = sc.nextInt();
        }}
        System.out.println("Read Matrix B"); for(int
        i=0;i<m2;i++){
        for(int j=0;j<n1;j++){
        System.out.print("B["+i+"]["+j+"]="); B[i][j] = sc.nextInt();
        }}
```

```
for(int i=0;i<m1;i++){
for(int j=0;j<n2;j++){
C[i][j]=0;
for(int k=0;k<n1;k++){
C[i][j] += A[i][k] * B[k][j];
}}
System.out.println("Matrix A"); for(int
i=0;i<m1;i++){
for(int j=0;j<n1;j++){
System.out.print(A[i][j]+"\t");
}
System.out.println();
}
System.out.println("Matrix B"); for(int
i=0;i<m2;i++){
for(int j=0;j<n2;j++){
System.out.print(B[i][j]+"\t");
}
System.out.println();
}]
System.out.println("Matrix C"); for(int
i=0;i<m1;i++){
for(int j=0;j<n2;j++){
System.out.print(C[i][j]+"\t");
}}}
```

45.Define Constructor? State its properties. Explain its uses with examples.

A constructor is a special method within a class that is automatically invoked when an object of that class is created. Its primary purpose is to initialize the newly created object, ensuring its attributes are set to appropriate initial values or a valid state.

Properties of Constructors:

- **Same Name as the Class:** A constructor always has the same name as the class it belongs to.
- **No Return Type:** Unlike regular methods, constructors do not have a return type, not even void.
- **Automatic Invocation:** Constructors are implicitly called when an object is instantiated using the new keyword (in languages like Java or C++).
- **Initialization Focus:** Their main role is to set the initial state of an object's attributes.
- **Cannot be Abstract or Static:** Constructors cannot be declared as abstract or static.

- **Can be Overloaded:** A class can have multiple constructors with different parameter lists, allowing for various ways to initialize an object.
- **Access Modifiers:** Constructors can have access modifiers (e.g., public, private, protected) to control their accessibility.

```
class Student{
int rollno;
String name;
float fee;
Student(int rollno,String name,float fee){
this.rollno=rollno;
this.name=name;
this.fee=fee;
}
void display(){System.out.println(rollno+" "+name+" "+fee);}
}

class TestThis2{
public static void main(String args[]){
Student s1=new Student(111,"ankit",5000f);
Student s2=new Student(112,"sumit",6000f);
s1.display();
s2.display();
}}
```

KTU SPECIAL
CONNECT WITH US

WHATSAPP GROUP

FOLLOW US NOW

TELEGRAM CHANNEL

SUBSCRIBE NOW

www.ktuspecial.in

SUBSCRIBE

FOLLOW US

Visit Website